

CSAD401 - OPERATING SYSTEMS

CO1 - ASSESSMENT TOOL - 1

Short Answer

1. Consider an Operating System managing multiple files and devices for concurrent users.
- (a) Categorize system calls used for file and device management.

Ans:

FILE MANAGEMENT SYSTEM CALLS:

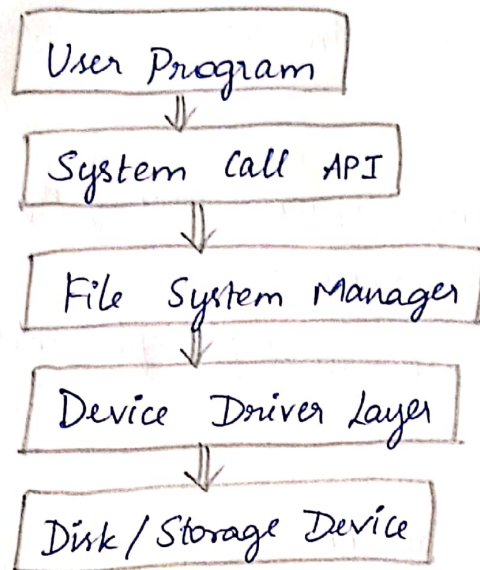
- `create()` - Creates a new file
- `open()` - Opens an existing file
- `read()` - Reads data from a file
- `write()` - Writes data into a file
- `close()` - Closes the opened file
- `delete()` - Deletes a file.

DEVICE MANAGEMENT SYSTEM CALLS:

- `request()` - Requests access to a device
- `release()` - Releases the device.
- `read()` - Reads data from a device
- `write()` - Sends data to a device.
- `ioctl()` - Controls device-specific operations.

(b) Illustrate how the OS handles file operations using a layered structure diagram.

Soln:



(c) Write a C-based system call sequence to open, read and close a file.

Soln:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd;
    char buffer[100];
    fd = open("Sample.txt", O_RDONLY);
    read(fd, buffer, sizeof(buffer));
    close(fd);
    return 0;
}
```

System Call Sequence:

open() → read() → close()

3. Assume a real-time Operating System used in an embedded system controlling industrial machinery.

(a) Identify the type of OS and explain why it is suitable for real-time applications.

Soln:

TYPE OF OS:

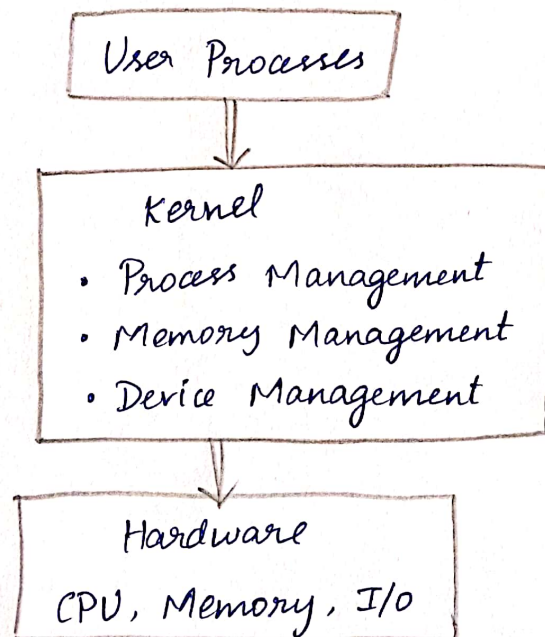
Real-Time Operating System (RTOS)

Why it is suitable:

- Provides fast response to events
- Meets strict timing deadlines
- Ensures reliable & predictable execution.
- Commonly used in industrial machines, robotics, medical devices & embedded systems.

(b) Draw a diagram representing the interaction between hardware, kernel & User processes.

Soln:



(c) List and explain system calls required for process synchronization.

Soln:

System Calls:

- *. `fork()` → Creates a new processes
- *. `wait()` → Parent waits until the child process finishes
- *. `signal()` → Sends a signal to another process
- *. `sem_wait()` → Locks (decrements) a semaphore for synchronization.
- *. `sem_post()` → Unlocks (increments) a semaphore after use.